

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 6

DATE : 07-AUG-2026

PAGE : 1 OF 25

TODAY'S TOPIC: GITHUB: FROM LOCAL GIT TO REMOTE COLLABORATION

1 What is GitHub?



- GitHub is a cloud platform to host Git repositories.
- It helps developers collaborate, track changes and build amazing software together.

2 Git vs GitHub

Git

- Distributed Version Control System
- Works locally

GitHub

- Cloud platform
- Hosts Git repositories
- Enables collaboration

VS

3 Create a GitHub Account

- Go to github.com
- Click on Sign up
- Enter your details
- Verify your email
- Your account is ready!



4 GitHub Dashboard Overview

- Repositories
- Issues
- Pull requests
- Actions
- Projects
- Packages



5 Create Your First Repository

- Click 'New' repository
- Add name, description
- Choose Public or Private
- Click Create repository
- Done!



6 Public vs Private Repository

Public

- Visible to everyone
- Great for open source



Private

- Visible only to you and collaborators
- For personal or company projects



7 Clone a Repository

- Clone means copying a remote repo to local machine.

```
git clone <repo-url>
```



8 Connect Local Git to GitHub

- Link your local repo to GitHub using:

```
git remote add origin <repo-url>
```

- origin is the default name for remote.



9 Push Your First Project

- Push your local changes to GitHub.

```
git push -u origin main
```

- Your code is now live on GitHub!



10 Pull Changes

- Pull means downloading changes from remote repo.

```
git pull origin main
```

- Keeps your local repo up to date.



11 Fetch vs Pull

Fetch

- Downloads changes but doesn't merge
- Safe to review first

Pull

- Downloads and merges changes
- Updates working directory



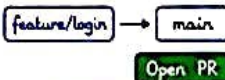
12 Branches in GitHub

- Branches help to work on features separately.
- Default branch is usually main or master.



13 Pull Requests (PR)

- PR is a request to merge your changes into another branch.
- Enables code review and discussion.



14 Fork vs Clone

Fork

- Creates a copy in your own GitHub account



Clone

- Copies the repo to your local machine



15 Issues & Labels

- Issues track tasks, bugs, or enhancements.
- Labels help categorize issues.



16 GitHub README.md

- README is the first thing people see.
- Add project info, setup steps, usage, badges, etc.



17 .gitignore File

- Tells Git which files or folders to ignore.
- Example: node_modules/.env, *.log



18 GitHub Releases & Tags

- Use tags to mark important versions.
- Releases provide release notes and binaries.



19 GitHub Actions (Introduction)

- Automate your workflows.
- Build, test, deploy using CI/CD
- YAML based configurations.



20 GitHub Security Features

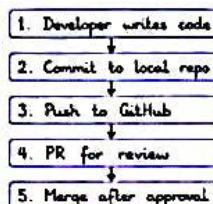
- Dependabot alerts
- Code scanning
- Secret scanning
- 2FA & Security settings



21 Best Practices

- ✓ Write clear commit messages
- ✓ Use .gitignore
- ✓ Keep PRs small and focused
- ✓ Review code before merging
- ✓ Keep main branch stable

22 GitHub Workflow (Dev → GitHub → Team)



23 Common Interview Questions

- What is GitHub?
- Git vs GitHub?
- What is a Pull Request?
- Fork vs Clone?
- What is .gitignore?
- Fetch vs Pull?
- How to revert a commit?



24 Real-time Project Workflow

- 1 Create repo on GitHub
- 2 Clone to local machine
- 3 Create a new branch
- 4 Make changes & commit
- 5 Push branch to GitHub
- 6 Open PR & merge

25 Day 6 Summary & Revision

- Learned GitHub basics & workflow
- Understand collaboration features
- Ready for advanced CI/CD with GitHub Actions!



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 6

DATE : 07-AUG-2026

PAGE : 2 OF 25

TODAY'S TOPIC: GITHUB: FROM LOCAL GIT TO REMOTE COLLABORATION

11 Fetch vs Pull

Fetch

- Downloads changes from remote repo but doesn't merge.
- Safe to review first.

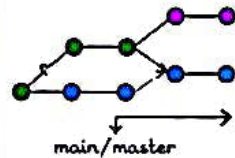
Pull

- Downloads and merges changes from remote.
- Updates your working directory.



12 Branches in GitHub

- Branches help you work on features separately.
- Two types:
 1. Main (Default) Branch - usually main or master
 2. Feature Branch - created for specific work



13 Pull Requests (PR)

- PR is used to merge changes from a feature branch to main branch.
- Enables code review, discussion and collaboration.



14 Fork vs Clone

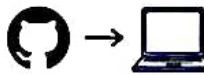
Fork

- Creates a copy of someone else's repo in your GitHub account.



Clone

- Copies a repo to your local machine.



15 Issues & Labels

- Issues track tasks, bugs, features.
- Labels help categorize and prioritize.

Example Labels:



16 GitHub README.md

- README.md is the first thing people see.
- It describes your project, installation, usage, etc.
- Good README = More stars ★

```
README.md
# Project Title
## Description
## Installation
## Usage
## Contributing
## License
```

17 .gitignore File

- Tells Git which files or folders to ignore.
- Keeps your repo clean and secure.
- Example:

```
.gitignore
node_modules/
.env
log
dist/
.DS_Store
```

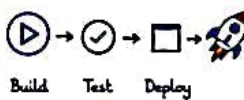
18 GitHub Releases & Tags

- Tags are used to mark important points (v1.0.0).
- Releases provide release notes and binaries for users.



19 GitHub Actions (Introduction)

- CI/CD tool by GitHub.
- Automate build, test, and deployment.
- Workflows are written in YAML files.



20 GitHub Security Features

- ✓ Dependabot alerts (keeps dependencies safe)
- ✓ Code scanning (find vulnerabilities)
- ✓ Secret scanning (detects leaked secrets)
- ✓ 2FA & Security settings (extra protection)



21 Best Practices

- ✓ Write clear commit messages
- ✓ Use .gitignore
- ✓ Keep PRs small and focused
- ✓ Review code before merging
- ✓ Keep main branch protected
- ✓ Update README



22 GitHub Workflow (Developer → GitHub → Team)

1. Developer creates feature branch
2. Make changes & commit
3. Push branch to GitHub
4. Open Pull Request
5. Code Review by Team
6. Approve & Merge to Main



23 Common Interview Questions

- What is GitHub?
- Difference between Git and GitHub?
- What is a Pull Request?
- Fork vs Clone?
- What is .gitignore?
- Fetch vs Pull?
- What is GitHub Actions?



24 Real-time Project Workflow

1. Create repo on GitHub
2. Clone to local machine
3. Create a new branch
4. Make changes & commit
5. Push branch to GitHub
6. Open PR & merge



25 Day 6 Summary & Revision

- ★ Understood GitHub and its features.
- ★ Learned collaboration with PRs.
- ★ Explored Actions, Issues, Releases and Security.
- ★ One step closer to DevOps mastery!



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 6

DATE : 07-AUG-2026

PAGE : 3 OF 25

TODAY'S TOPIC : GITHUB: FROM LOCAL GIT TO REMOTE COLLABORATION

1 What is GitHub?

- GitHub is a cloud platform to host Git repositories.



- It helps developers collaborate, track changes and build amazing software together.



2 Git vs GitHub

Git

- Distributed Version Control System.
- Works locally

VS

GitHub

- Cloud platform.
- Hosts Git repositories.
- Enables collaboration.

3 Create a GitHub Account

- Go to github.com
- Click on Sign up
- Enter details, verify email.
- Your account is ready! 🎉



4 GitHub Dashboard Overview

- Repositories
- Issues
- Pull requests
- Actions
- Projects
- Packages
- Settings



5 Create Your First Repository

- Click 'New' repository
- Add name, description
- Choose Public or Private
- Click 'Create repository'
- Done! 🎉



6 Public vs Private Repository

Public

- Visible to everyone
- Great for open source



Private

- Visible only to you and collaborators
- For personal or company projects



7 Clone a Repository

- Clone means copying a remote repo to local machine.

```
$ git clone https://github.com/user/repo.git
```



8 Connect Local Git to GitHub

- Link your local repository to GitHub.

```
$ git remote add origin <repo-url>
```

- origin is the default remote name.



9 Push Your First Project

- Push your local changes to GitHub.

```
$ git push -u origin main
```

- Your code is now live on GitHub!



10 Pull Changes

- Pull means downloading changes from remote repo.

```
$ git pull origin main
```

- Keeps your local repo up to date.



11 Fetch vs Pull

Fetch

- Downloads changes but doesn't merge.
- Safe to review first.

Pull

- Downloads and merges changes.
- Updates working directory.



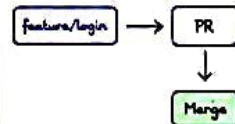
12 Branches in GitHub

- Branches help to work on features separately.
- Default branch is usually main.
- Create branch from main or other branch.



13 Pull Requests (PR)

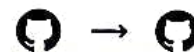
- PR is a request to merge your changes into another branch.
- Enables code review and discussion.



14 Fork vs Clone

Fork

- Creates a copy of someone else's repo in your GitHub account.



Clone

- Copies a repo to your local machine.



15 Issues & Labels

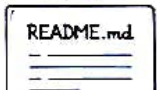
- Issues track tasks, bugs and features.
- Labels help categorize and priorities

Example Labels:



16 GitHub README.md

- README.md is the first thing people see.
- It describes your project, installation, usage, etc.
- Good README = More stars ⭐



17 .gitignore File

- Tells Git which files or folders to ignore.
- Keeps repo clean and secure.
- Example:



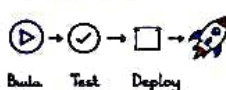
18 GitHub Releases & Tags

- Tags are used to mark important points (v1.0.0).
- Releases provide release notes and binaries for users.



19 GitHub Actions (Introduction)

- CI/CD tool by GitHub.
- Automate build, test and deployment.
- Workflows are written in YAML files.



20 GitHub Security Features

- ✓ Dependabot alerts
- ✓ Code scanning
- ✓ Secret scanning
- ✓ 2FA & Security settings



21 Best Practices

- ✓ Write clear commit messages
- ✓ Use .gitignore
- ✓ Keep PRs small and focused
- ✓ Review code before merging
- ✓ Keep main branch stable
- ✓ Update README



22 GitHub Workflow (Developer → GitHub → Team)

1. Developer writes code
2. Commit to local repo
3. Push to GitHub
4. Create Pull Request
5. Code Review by Team
6. Merge after approval

23 Common Interview Questions

- What is GitHub?
- Difference between Git and GitHub?
- What is a Pull Request?
- Fork vs Clone?
- What is .gitignore?
- Fetch vs Pull?
- What is GitHub Actions?



24 Real-time Project Workflow

1. Create repo on GitHub
2. Clone to local machine
3. Create a new branch
4. Make changes & commit
5. Push branch to GitHub
6. Open PR & merge



25 Day 6 Summary & Revision

- ★ Learned GitHub core features.
- ★ Understood collaboration with teams
- ★ Explored Actions, Issues, Releases & Security.
- ★ One step closer to DevOps mastery! 🌟



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 6

DATE : 07-AUG-2026

PAGE : 5 OF 25

TODAY'S TOPIC: GITHUB: FROM LOCAL GIT TO REMOTE COLLABORATION

1 What is GitHub?

- GitHub is a cloud platform to host Git repositories.
- It helps developers collaborate, track changes and build amazing software together.



2 Git vs GitHub

- Distributed Version Control System.
- Works locally.

VS

- Cloud platform.
- Hosts Git repositories.
- Enables collaboration.



3 Create a GitHub Account

- Go to github.com
- Click Sign up.
- Enter details, verify email.
- Your account is ready! 🎉



4 GitHub Dashboard Overview

- Repositories
- Issues
- Pull requests
- Actions
- Projects
- Packages
- Settings



5 Create Your First Repository

- Click 'New' repository
- Add name, description
- Choose Public
- Click Create repository
- Done! 🎉



6 Public vs Private Repository

Public

- Visible to everyone.
- Great for open source.



Private

- Visible only to you and collaborators.
- For personal or company projects.



7 Clone a Repository

- Clone means copying a remote repo to local machine.

```
$ git clone https://github.com/user/repo.git
```



8 Connect Local Git to GitHub

- Link your local repo to remote.

```
$ git remote add origin <repo-url>
```

- origin is the default remote name.



9 Push Your First Project

- Push your local changes to GitHub.

```
$ git push -u origin main
```

- Your code is now live on GitHub!



10 Pull Changes

- Pull means downloading changes from remote.

```
$ git pull origin main
```

- Keeps your local repo up to date.



11 Fetch vs Pull

Fetch

- Downloads changes but doesn't merge.
- Safe to review first.

Pull

- Downloads and merges changes.
- Updates working directory



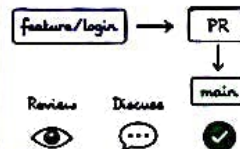
12 Branches in GitHub

- Branches help to work on features separately.
- Default branch is usually main.
- Create branch from main or other branch.



13 Pull Requests (PR)

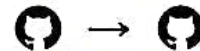
- PR is a request to merge your changes into another branch.
- Enables code review and discussion.



14 Fork vs Clone

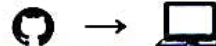
Fork

- Creates a copy of someone else's repo in your account.



Clone

- Copies a repo to your local machine.



15 Issues & Labels

- Issues track tasks, bugs and features.
- Labels help categorize and prioritize.

Example Labels:



16 GitHub README.md

- README.md is the first thing people see.
- It describes your project, installation, usage, etc.
- Good README = More stars ⭐



17 .gitignore File

- Tells Git which files or folders to ignore.
- Keeps repo clean and secure.
- Example:

```
.gitignore
node_modules/
.env
*.log
dist/
```

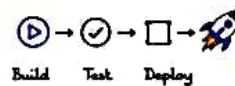
18 GitHub Releases & Tags

- Tags are used to mark important points (v1.0.0).
- Releases provide release notes and binaries for users.



19 GitHub Actions (Introduction)

- CI/CD tool by GitHub.
- Automate build, test and deployment.
- Workflows are written in YAML files.



20 GitHub Security Features

- ☑ Dependabot alerts (vulnerabilities)
- ☑ Code scanning
- ☑ Secret scanning
- ☑ 2FA & Security settings (extra protection)



21 Best Practices

- ☑ Write clear commit messages.
- ☑ Use .gitignore.
- ☑ Keep PRs small and focused.
- ☑ Review code before merging
- ☑ Keep main branch
- ☑ Update README



22 GitHub Workflow (Developer -> GitHub -> Team)

1. Developer writes code
2. Commit to local repo
3. Push to GitHub
4. Create Pull Request
5. Code Review by Team
6. Approve & Merge
7. Deploy / Release



23 Common Interview Questions

- What is GitHub?
- Difference between Git and GitHub?
- What is a Pull Request?
- Fork vs Clone?
- What is .gitignore?
- Fetch vs Pull?
- What is GitHub Actions?



24 Real-time Project Workflow

1. Create repo on GitHub
2. Clone to local machine
3. Create a new branch
4. Make changes & commit
5. Push branch to GitHub
6. Open PR & merge



25 Day 6 Summary & Revision

- ★ Learned GitHub core concepts.
- ★ Understood collaboration using PRs, Issues, Labels & Actions.
- ★ One step closer to DevOps mastery!



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 6

DATE : 07-AUG-2026

PAGE : 6 OF 25

TODAY'S TOPIC: GITHUB: FROM LOCAL GIT TO REMOTE COLLABORATION

1 What is GitHub?

- GitHub is a cloud platform to host Git repositories.
- It helps developers collaborate, track changes and build amazing software together.



2 Git vs GitHub

Git

- Distributed Version Control System.
- Works locally.

VS

GitHub

- Cloud platform.
- Hosts Git repositories.
- Enables collaboration.

3 Create a GitHub Account

- Go to github.com
- Click Sign up.
- Enter details and verify email.
- Your account is ready! 🎉

Join GitHub

Username

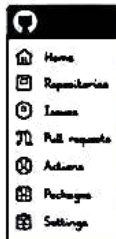
Email

Password

Sign up

4 GitHub Dashboard Overview

- Repositories
- Issues
- Pull requests
- Actions
- Projects
- Packages
- Settings



5 Create Your First Repository

- Click 'New' repository.
- Add name, description.
- Choose Public or Private.
- Click Create repository.
- Done! 🎉

New repository

Repo name

my first repo

Description (optional)

Public

Private

Create repository

6 Public vs Private Repository

Public

- Visible to everyone.
- Great for open source.



Private

- Visible only to you and collaborators
- For personal or company projects.



7 Clone a Repository

- Clone means copying a remote repo to local machine.

```
$ git clone https://github.com/user/repo.git
```



8 Connect Local Git to GitHub

- Link your local repo to GitHub.

```
$ git remote add origin <repo-url>
```

- origin is the default remote name.



9 Push Your First Project

- Push your local changes to GitHub.

```
$ git push -u origin main
```

- Your code is now live on GitHub!



10 Pull Changes

- Pull means downloading changes from remote repo.

```
$ git pull origin main
```

- Keeps your local repo up to date.



11 Fetch vs Pull

Fetch

- Downloads changes but doesn't merge.
- Safe to review first.

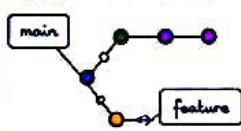
Pull

- Downloads and merges changes.
- Updates working directory.



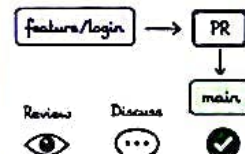
12 Branches in GitHub

- Branches help to work on features separately.
- Default branch is usually main.
- Create branch from main or other branch.



13 Pull Requests (PR)

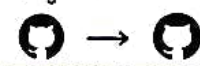
- PR is a request to merge your changes into another branch.
- Enables code review and discussion.



14 Fork vs Clone

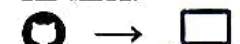
Fork

- Creates a copy of someone else's repo in your GitHub account.



Clone

- Copies a repo to your local machine.



15 Issues & Labels

- Issues track tasks, bugs and features.
- Labels help categorize and prioritize.

Example Labels:



16 GitHub README.md

- README.md is the first thing people see.
- It describes your project, installation, usage, etc.
- Good README = More ⭐



17 .gitignore File

- Tells Git which files or folders to ignore.
- Keeps repo clean and secure.
- Example:

```
.gitignore
node_modules/
.env
*.log
dist/
```

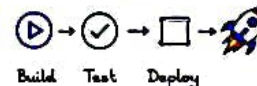
18 GitHub Releases & Tags

- Tags are used to mark important points (v1.0.0).
- Releases provide release notes and binaries for users.



19 GitHub Actions (Introduction)

- CI/CD tool by GitHub.
- Automate build, test and deployment.
- Workflows are written in YAML files.



20 GitHub Security Features

- ✓ Dependabot alerts (vulnerabilities)
- ✓ Code scanning
- ✓ Secret scanning (detects leaked secrets)
- ✓ 2FA & Security settings (extra protection)



21 Best Practices

- ✓ Write clear commit messages
- ✓ Use .gitignore
- ✓ Keep PRs small and focused
- ✓ Review code before merging
- ✓ Keep main branch stable
- ✓ Update README regularly



22 GitHub Workflow (Developer -> GitHub -> Team)

1. Developer writes code
2. Commit to local repo
3. Push to GitHub
4. Create Pull Request
5. Code Review by Team
6. Approve & Merge
7. Deploy / Release



23 Common Interview Questions

- What is GitHub?
- Difference between Git and GitHub?
- What is a Pull Request?
- Fork vs Clone?
- What is .gitignore?
- Fetch vs Pull?
- What is GitHub Actions?



24 Real-time Project Workflow

1. Create repo on GitHub
2. Clone to local machine
3. Create a new branch
4. Make changes & commit
5. Push branch to GitHub
6. Open PR & merge



25 Day 6 Summary & Revision

- ★ Learned GitHub from basics to collaboration.
- ★ Explored features like Issues, Actions, Releases and Security.
- ★ One step closer to DevOps mastery! 🌟



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 6

DATE : 07-AUG-2026

PAGE : 7 OF 25

TODAY'S TOPIC : GITHUB : FROM LOCAL GIT TO REMOTE COLLABORATION

16 GitHub README.md

- README.md is the first thing people see.
- It describes your project, installation, usage, etc.
- Good README = More stars ★

```
README.md
# Project Title
## Description
## Installation
## Usage
## Contributing
## License
```

17 .gitignore File

- Tells Git which files or folders to ignore.
- Keeps repo clean and secure.
- Example :

```
.gitignore
node_modules/
.env
*.log
dist/
.DS_Store
```

18 GitHub Releases & Tags

- Tags are used to mark important points (v1.0.0).
- Releases provide release notes and binaries for users.

v1.0.0

Releases

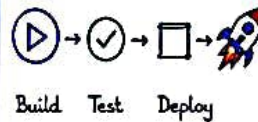
✓ v1.0.0 Latest

Release notes...

> Assets
app.zip 2.1 MB

19 GitHub Actions (Introduction)

- CI/CD tool by GitHub.
- Automate build, test and deployment.
- Workflows are written in YAML files.



20 GitHub Security Features

- ✓ Dependabot alerts (vulnerabilities)
- ✓ Code scanning (find vulnerabilities)
- ✓ Secret scanning (detects leaked secrets)
- ✓ 2FA & Security settings (extra protection)



21 Best Practices

- ★ Write clear commit messages
- ★ Use .gitignore
- ★ Keep PRs small and focused
- ★ Review code before merging
- ★ Keep main branch protected
- ★ Update README regularly



22 GitHub Workflow (Developer → GitHub → Team)

1. Developer writes code
2. Commit to local repo
3. Push to GitHub
4. Create Pull Request
5. Code Review by Team
6. Approve & Merge
7. Deploy / Release



23 Common Interview Questions

- What is GitHub?
- Difference between Git and GitHub?
- What is a Pull Request?
- Fork vs Clone?
- What is .gitignore?
- Fetch vs Pull?
- What is GitHub Actions?
- How do you create a repository?
- How do you resolve merge conflicts?



24 Real-time Project Workflow

- 1 Create repo on GitHub
- 2 Clone to local machine
- 3 Create a new branch
- 4 Make changes & commit
- 5 Push branch to GitHub
- 6 Open Pull Request
- 7 Code review & feedback
- 8 Merge to main branch
- 9 Deploy application



25 Day 6 Summary & Revision

- ★ Learned about GitHub and its core features.
- ★ Understood repositories, branches, PRs, issues and labels.
- ★ Explored GitHub Actions and security features.
- ★ One step closer to DevOps mastery!



- NOTE :**
- ★ Practice daily.
 - ★ Build small projects.
 - ★ Collaborate and contribute.
 - ★ Consistency makes you a DevOps Hero!

Pro Tip

Use meaningful commit messages and follow a clean workflow. It makes teamwork easy and professional!

Reminder

Tomorrow we move to GitHub Actions (CI/CD) - stay tuned!

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 6

DATE : 07-AUG-2026

PAGE : 8 OF 25

TODAY'S TOPIC : GITHUB : FROM LOCAL GIT TO REMOTE COLLABORATION

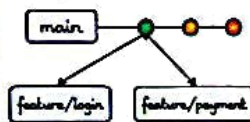
26 .gitignore Best Practices

- Keep it at the root of the repository.
- Ignore files, not folders.
- Add commonly ignored patterns.
- Update when new files are added.
- Use global gitignore for OS/IDE files.



27 GitHub Branching Strategy

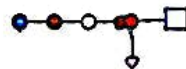
- main (or master) is the production branch.
- Create feature branches for new work.
- Use branches for bug fixes and releases.
- Merge using Pull Requests.



28 Merge vs Rebase

Merge:

- Preserves history.
- Creates a merge commit.



Rebase:

- Rewrites history.
- Keeps project history clean.



29 Pull Request Best Practices

- ✓ Small and focused PRs.
- ✓ Clear title and description.
- ✓ Link related issues.
- ✓ Add screenshots if required.
- ✓ Request review from the right people.
- ✓ Address review comments.



30 Code Review Checklist

- ✓ Code follows standards.
- ✓ Easy to read and understand.
- ✓ No duplicate code.
- ✓ Proper error handling.
- ✓ Adequate test cases.
- ✓ No hardcoded secrets.
- ✓ Documentation updated.



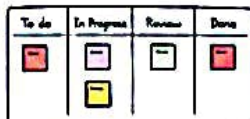
31 GitHub Issues

- Used to track tasks, bugs, enhancements, and features.
- Can be linked to PRs.
- Use Labels, Assignees, Milestones.
- Helps in project planning.



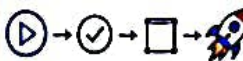
32 GitHub Projects

- Kanban style project management.
- Columns: To do, In Progress, Review, Done.
- Track issues and PRs.
- Visualize the workflow.
- Great for Agile teams.



33 GitHub Actions (Introduction)

- CI/CD tool by GitHub.
- Automate build, test and deployment.
- Workflows written in YAML files.
- Triggered by events: push, PR, schedule, etc.



34 GitHub Actions Workflow Anatomy

- name: Workflow name
- on: Trigger (push, pr, etc.)
- jobs: Define jobs
- runs-on: Runner (ubuntu-latest)
- steps: List of steps
- uses/run: Actions or shell commands

```
name: CI Workflow
on: [push, pullrequest]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Hello DevOps!"
```

35 GitHub Actions Benefits

- ★ Easy automation with YAML.
- ★ Huge marketplace of actions.
- ★ Tight integration with GitHub.
- ★ Saves time and reduces manual errors.



36 Secrets in GitHub Actions

- Store sensitive data in Secrets.
- Example: AWS keys, tokens, passwords.
- Add secrets in Repository Settings.
- Access using `{{ secrets.NAME }}`



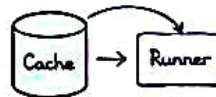
37 Runners in GitHub Actions

- GitHub-hosted runners (Ubuntu, Windows, macOS).
- Self-hosted runners (your own servers).
- Choose based on requirements.



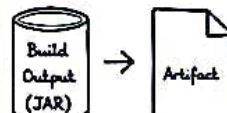
38 Caching in GitHub Actions

- Cache dependencies to speed up workflows.
- Use actions/cache.
- Example: Node modules, Maven, pip, etc.



39 Artifacts in GitHub Actions

- Store build outputs.
- Share files between jobs.
- Example: JAR, Docker images, test reports.



40 Deploy with GitHub Actions

- Deploy to servers, Docker, Kubernetes, AWS, etc.
- Example: Deploy to AWS S3 / EC2.



TODAY WE LEARNED

- ✓ GitHub advanced concepts.
- ✓ Branching strategy and best practices.
- ✓ Pull Requests and Code Review.
- ✓ Issues, Projects and Collaboration.
- ✓ Introduction to GitHub Actions CI/CD.
- ✓ Secrets, Runners, Caching, Artifacts and Deployment.



QUICK RECAP

- Local Git → Commit changes
- Push to GitHub → Remote collaboration
- Branches → Work in parallel
- Pull Requests → Review & merge
- Actions → Automate everything!



DAY 6

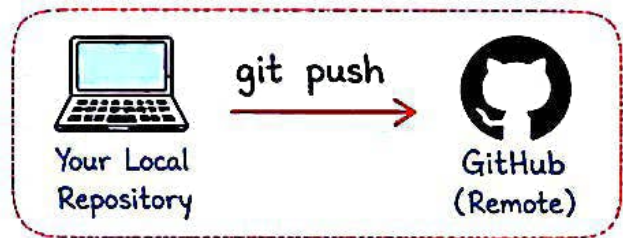
Page 9 of 25

GIT & GITHUB FUNDAMENTALS

9. PUSH YOUR FIRST PROJECT



- ★ What does "git push" do?
It uploads your local commits to a remote repository (like GitHub).



- ★ Command to push

```
git push -u origin main
```

-u : sets upstream tracking
origin : remote name (default)
main : branch name

- ★ Steps to push your first project

- ① Create a repo on GitHub (Don't initialize with README)
- ② Copy the repo URL (HTTPS or SSH)
- ③ In your local project folder:

```
git remote add origin <repo_url>  
git push -u origin main
```

Example
origin = <https://github.com/username/my-project.git>

- ★ Verify on GitHub

Refresh your repository page → you should see your files and commits. ✓

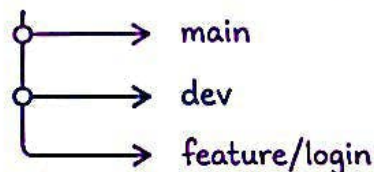
- ★ Common Push Errors & Fixes

- Permission denied (publickey) → Add SSH key to GitHub
- ![rejected] → Pull latest changes and try again
- Repository not found → Check repo URL and access

💡 Always pull before pushing if others are working on the same repo.

- ★ Best Practices After First Push

- ✓ Keep your README.md updated
- ✓ Push meaningful commits regularly
- ✓ Create .gitignore to avoid unwanted files
- ✓ Follow a consistent branch strategy

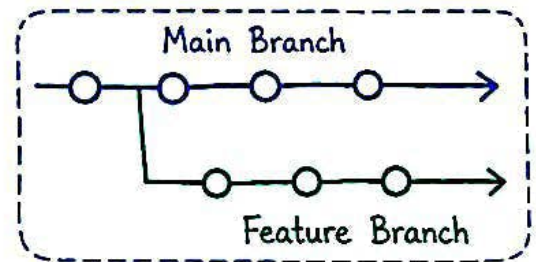


10. WORKING WITH BRANCHES



Why Branches?

- ✓ Branches help you work on features without affecting the main code.
- ✓ They allow safe experimentation.
- ✓ Once ready, you merge the changes to main.



Common Branch Commands

- | | |
|--|-------------------------------------|
| ① <code>git branch</code> | → List all branches |
| ② <code>git branch <branch_name></code> | → Create a new branch |
| ③ <code>git checkout <branch_name></code> | → Switch to a branch |
| ④ <code>git checkout -b <branch_name></code> | → Create and switch to a new branch |
| ⑤ <code>git branch -d <branch_name></code> | → Delete a branch (safe delete) |
| ⑥ <code>git branch -D <branch_name></code> | → Force delete a branch |



Example Workflow

main → Create Feature Branch → Make Changes → Commit → Merge to main

```

# Create and switch to feature branch
git checkout -b feature/login
# Make changes and commit
git add .
git commit -m "Add login page"
# Switch to main and merge
git checkout main
git merge feature/login
  
```



Tip:

Always keep your main branch clean and stable.

Do your work in feature branches.



Best Practices

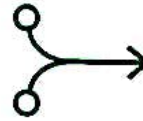
- ✓ Name branches meaningfully (e.g., feature/login, bugfix/header)
- ✓ Keep branches small and focused
- ✓ Regularly pull the latest changes from main
- ✓ Delete branches after merging

Naming Convention

```

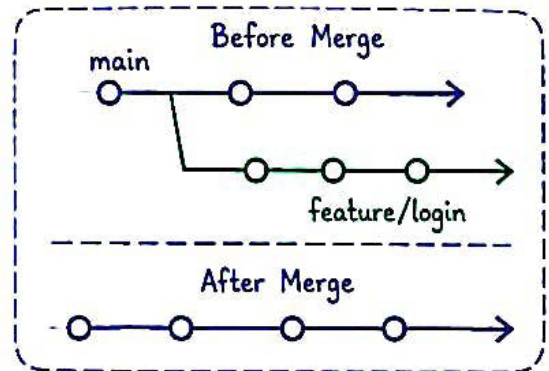
feature/*
bugfix/*
hotfix/*
  
```


11. MERGING BRANCHES



What is Merge?

Merging combines the changes from one branch into another. It integrates your work and keeps the project up to date.



Common Merge Scenarios

- ① Merge feature branch into main
- ② Resolve conflicts during merge
- ③ Merge using command line
- ④ Merge using GitHub (Pull Request)

Merge Commands

```
git checkout main      # Switch to main branch
git pull               # Get latest changes
git merge <branch_name> # Merge the branch
git branch -d <branch_name> # Delete branch after merge
```

Example

```
# Merge feature/login into main
git checkout main
git pull
git merge feature/login
git push origin main
git branch -d feature/login
```

Handling Merge Conflicts

- ① Git will stop and show conflicting files.
- ② Open the file and look for conflict markers.
- ③ Edit the file and keep the correct changes.
- ④ Save the file.
- ⑤ Run: `git add <file>`
- ⑥ Complete the merge: `git commit`
- ⑦ Push the changes.

Conflict Example in a file

```
<<<<<< HEAD
This is main branch change
=====
This is feature branch change
>>>>>> feature/login
```

These marks show conflict areas

Best Practices

- ✓ Always pull latest changes before merging.
- ✓ Resolve conflicts carefully and test your code.
- ✓ Delete merged branches to keep repo clean.
- ✓ Use Pull Request for team collaboration.



Tip:

Communicate with your team before merging to avoid conflicts.

DAY 6

GIT & GITHUB FUNDAMENTALS

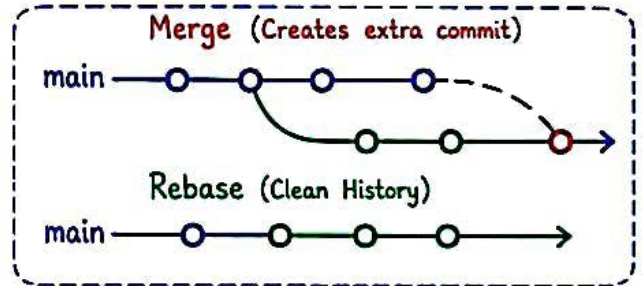
Page 12 of 25

12. REBASING BASICS



What is Rebase?

- ✓ Rebase moves or combines a series of commits to a new base commit.
- ✓ It creates a clean, linear project history.
- ✓ Useful before pushing to remote repository.



Common Rebase Commands

- `git checkout <branch>` # Switch to the branch
- `git fetch origin` # Get latest from remote
- `git rebase origin/main` # Rebase your branch on main
- `git rebase -i HEAD~n` # Interactive rebase (advanced)
- `git rebase --abort` # Cancel the rebase
- `git rebase --continue` # Continue after resolving conflict

Tip:

Rebase is best used in feature branches before merging into main or before pushing to remote.

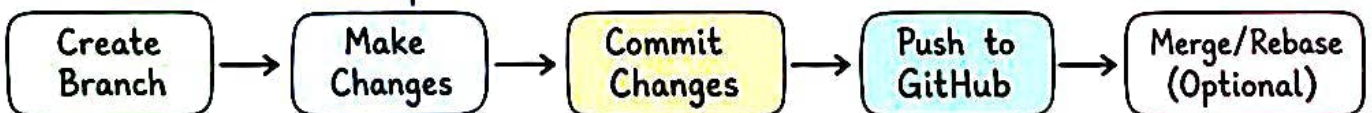
Merge vs Rebase – When to Use?

Feature	Merge	Rebase
History	Creates extra merge commits	Keeps history clean & linear
Use Case	Team collaboration (public)	Personal branches (before push)
Risk	Low	High (rewrites history)
Best For	Stable branches	Feature development



Keep your history clean and easy to understand.

Git Workflow Recap



Best Practices

- ✓ Always pull latest changes before working.
- ✓ Use meaningful commit messages.
- ✓ Prefer rebase for clean history (when applicable).
- ✓ Never commit directly to main branch.
- ✓ Review your changes before merging.

Commit Message Format

<Type>: <Short description>
<More details if needed>

Example:

feat: add login page

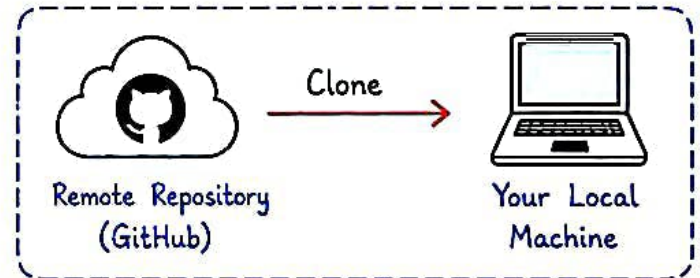


13. CLONING REPOSITORIES



★ What is Cloning?

Cloning creates a copy of an existing remote repository on your local machine. It includes all the files, history, and branches.



★ Clone Command

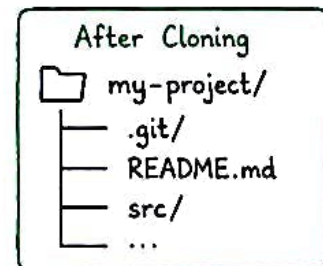
```
git clone <repository-url>
```

Example :

```
git clone https://github.com/username/my-project.git
```

★ What Happens When You Clone?

- ① A new folder is created with the repository name.
- ② All files and folders are downloaded.
- ③ Git history is available locally.
- ④ Remote named "origin" is set automatically.



★ Verify Cloned Repository

<code>cd my-project</code>	→ Go inside the project folder
<code>git status</code>	→ Check the status
<code>git branch</code>	→ List all branches
<code>git remote -v</code>	→ Verify remote URL

 **Tip:**
Always check the remote URL after cloning to make sure it's correct.

★ Common Use Cases

- ✓ Work on an open source project
- ✓ Collaborate with a team
- ✓ Fork and clone your own copy
- ✓ Backup or analyze code

Pro Tip

After cloning, always pull the latest changes before starting your work.

```
git pull origin main
```



★ Best Practices

- ✓ Clone only the repositories you need.
- ✓ Keep your local repository updated.
- ✓ Use meaningful folder names.
- ✓ Avoid cloning large repos unnecessarily.
- ✓ Regularly fetch and pull changes.

Remember

Clone once, code many times!
Keep your local copy clean, updated, and organized.



GIT & GITHUB FUNDAMENTALS

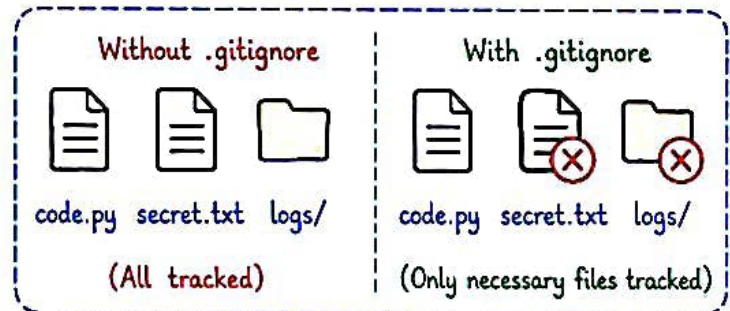
14. IGNORING FILES



★ What is .gitignore?

A .gitignore file tells Git which files and folders to ignore.

Ignored files will not be tracked or pushed to the remote repository.



★ Create .gitignore File

- ① Create a file named .gitignore in the root of your project.
- ② Add the files and folders you want to ignore.

Touch command
touch .gitignore

★ Example .gitignore Content

```
# Logs
*.log
# Environment files
.env
# OS generated files
.DS_Store
Thumbs.db
# Dependencies
node_modules/
__pycache__/
# IDE
.vscode/
.idea/
# Build files
dist/
build/
```

- Ignore all .log files
- Ignore environment variable files
- Ignore files generated by OS
- Ignore Node.js and Python cache
- Ignore VS Code and IntelliJ settings
- Ignore build and distribution folders



Tip:

Always keep .gitignore updated as your project grows. Never commit secrets or credentials!

Commonly Ignored

- .env
- .log
- node_modules/
- __pycache__/
- .vscode/
- dist/
- build/

★ Verify .gitignore

git status

- Ignored files will not appear in the list.
- Use "git check-ignore <file>" to check if a file is ignored.

★ Best Practices

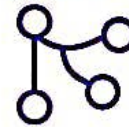
- ✓ Never commit sensitive information.
- ✓ Keep your repository clean and professional.
- ✓ Share only the required files.
- ✓ Review .gitignore before pushing your code.

Example Check

```
git check-ignore secret.txt
# Shows the pattern that ignores the file
```

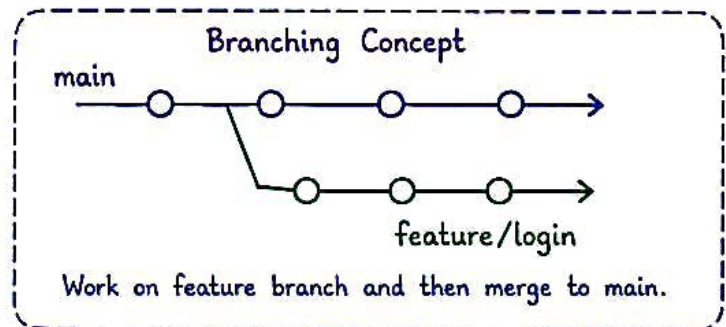

GIT & GITHUB FUNDAMENTALS

15. BRANCHES IN GIT



What is a Branch?

A branch is an independent line of development. It allows you to work on new features or bug fixes without affecting the main code.



Common Branch Commands

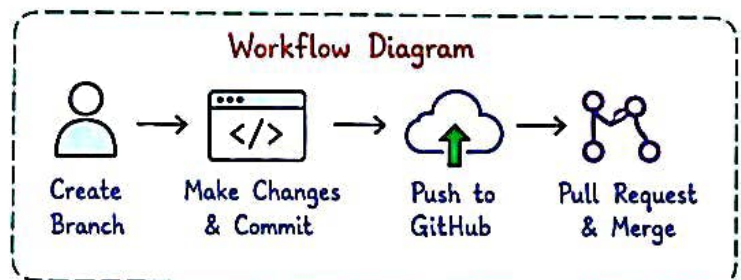
```
git branch                # List all branches
git branch <branch_name>  # Create a new branch
git checkout <branch_name> # Switch to branch
git switch <branch_name>   # (New command) Switch to branch
git branch -m <new_name>   # Rename a branch
git branch -d <branch_name> # Delete merged branch
git branch -D <branch_name> # Force delete branch
```

Example

```
git branch
git branch feature/signup
git checkout feature/signup
git add .
git commit -m "Add signup page"
```

Typical Branch Workflow

- 1 Create a new branch for the task.
- 2 Do the changes and commit in that branch.
- 3 Push the branch to remote.
- 4 Create a Pull Request.
- 5 After review, merge the branch to main.
- 6 Delete the branch (locally and remotely).



Best Practices for Branching

- ✓ Use meaningful branch names (e.g., feature/login, bugfix/header-issue).
- ✓ Keep your branch small and focused.
- ✓ Regularly pull the latest changes from main.
- ✓ Delete branches that are no longer needed.
- ✓ Follow your team's branching strategy (e.g., Git Flow).



Tip:

Keep your main branch stable.
All changes should come through Pull Requests.

Branch vs Main

Branch	Main
Used for development and testing	Always stable and production ready
Can be created and deleted	Usually always exists
Changes here are not final	Changes here are final and merged

Remember

Branches help in safe and collaborative development.
Work in branches,
merge with confidence!

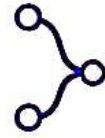


DAY 6

Page 16 of 25

GIT & GITHUB FUNDAMENTALS

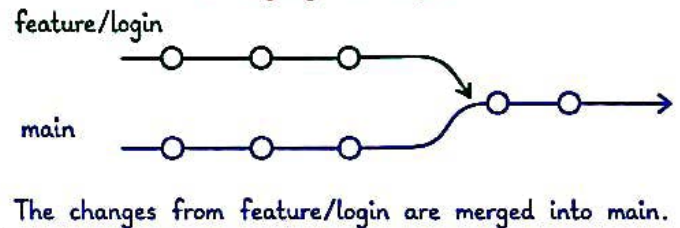
16. MERGING BRANCHES



What is Merging?

Merging is the process of combining changes from one branch into another. It helps to bring all the work together in a single branch.

Merging Concept



Merge Commands

```
git checkout main          # Switch to main branch
git pull origin main       # Get latest changes
git merge <branch_name>    # Merge branch into current branch
git branch -d <branch_name> # Delete merged branch
git branch -D <branch_name> # Force delete branch
```

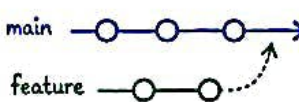
Example

```
git checkout main
git pull origin main
git merge feature/login
git branch -d feature/login
```

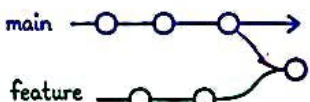
Types of Merge

- 1) Fast-Forward Merge : Moves the main pointer forward if no divergence.
- 2) Three-Way Merge : Creates a merge commit to combine histories.
- 3) Merge Conflict : Happens when changes conflict in the same file.

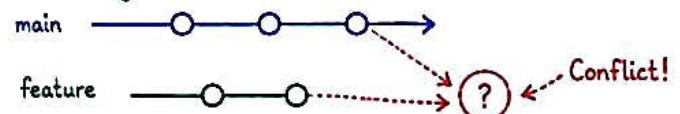
1) Fast-Forward Merge



2) Three-Way Merge



3) Merge Conflict



Handling Merge Conflicts

- 1) Git will mark the conflicting files.
- 2) Open the file and look for conflict markers.
- 3) Fix the conflicts manually.
- 4) Add the resolved files.
- 5) Commit the merge.

Conflict Markers in File

```
<<<<<<< HEAD
Your changes (main branch)
=====
Their changes (feature branch)
>>>>>> feature/login
```



Tip:

Discuss with your team before merging large changes.

Best Practices

- ✓ Always pull the latest changes before merging.
- ✓ Keep your branches updated and short-lived.
- ✓ Do code review before merging.
- ✓ Resolve conflicts carefully and test after merge.
- ✓ Delete the branch after successful merge.

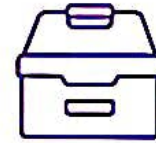
Remember

Good merging keeps your code base clean, stable, and team collaboration smooth!



GIT & GITHUB FUNDAMENTALS

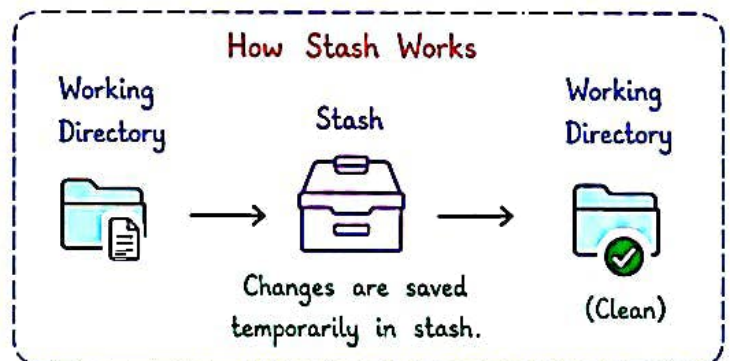
17. STASH IN GIT



★ What is Stash?

Stash is a way to temporarily save your changes (modified tracked files) without committing them.

It helps you to switch branches or work on something else and come back to your changes later.



★ Common Stash Commands

```

git stash          # Stash current changes
git stash list     # List all stashes
git stash apply    # Apply the latest stash (keeps stash)
git stash pop      # Apply the latest stash and drop it
git stash apply stash@{n} # Apply specific stash
git stash drop stash@{n} # Delete specific stash
git stash clear    # Delete all stashes
    
```

Example Workflow

- ① Make some changes
- ② git stash
- ③ Switch to another branch
- ④ Do your work
- ⑤ git stash pop
- ⑥ Your changes are back!

★ When to Use Stash?

- ✓ When you are not ready to commit.
- ✓ When you want to switch branches quickly.
- ✓ When you want to pull latest changes from remote.
- ✓ When working on multiple tasks.



Tip:

Stash is only for tracked files.
Untracked files are not stashed.
→ git stash -u (include untracked files)

★ Practical Example

- ① # Make some changes in files
- ② git stash # Save changes to stash
- ③ git checkout feature/login # Switch to another branch
- ④ # Do some work...
- ⑤ git stash pop # Bring back your changes

Stash List Example

```

$ git stash list
stash@{0}: WIP on main: a1b2c3d Fix UI
stash@{1}: WIP on feature/login: f3d4e5f Add login
stash@{2}: WIP on main: 9a8b7c6 Update README
    
```

★ Best Practices

- ✓ Always give meaningful commit messages after applying stash.
- ✓ Use "stash pop" instead of "apply" to keep things clean.
- ✓ Clear unused stashes regularly.
- ✓ Don't rely on stash for long-term storage.

Remember

Stash is temporary.
Clean commits make your repo professional!



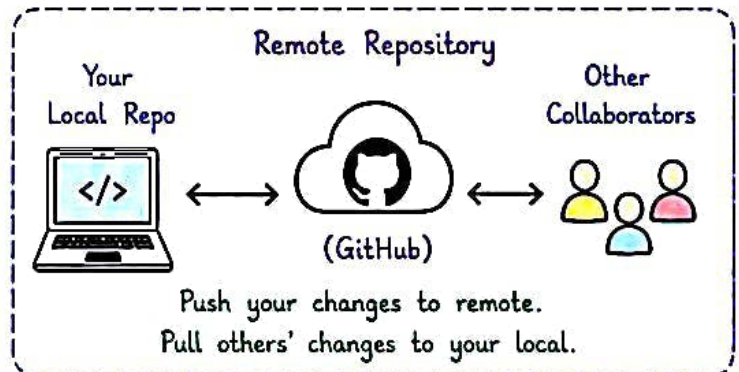
GIT & GITHUB FUNDAMENTALS

18. REMOTE REPOSITORIES



★ What is a Remote Repository?

A remote repository is a version of your project that is hosted on the internet (e.g., GitHub, GitLab, Bitbucket). It allows you to collaborate with others and keep your code safe.



★ Adding a Remote

```
git remote add origin <remote-url>
```

Example :

```
git remote add origin https://github.com/username/repo.git
```

★ Common Remote Commands

<code>git remote -v</code>	# List remote repositories
<code>git remote add origin <url></code>	# Add a new remote
<code>git remote remove origin</code>	# Remove a remote
<code>git fetch origin</code>	# Download changes from remote
<code>git pull origin <branch></code>	# Fetch and merge changes
<code>git push origin <branch></code>	# Push changes to remote

Example Workflow

- ① Create or clone a repository
- ② Make changes in your local repo
- ③ Commit your changes
- ④ Push changes to remote
- ⑤ Collaborators pull the changes
- ⑥ Everyone is in sync!

★ Cloning a Remote Repository

```
git clone <remote-url>
```

Example :

```
git clone https://github.com/username/repo.git
```

★ Tracking Remote Branches

<code>git branch -r</code>	# List remote branches
<code>git branch -a</code>	# List all branches
<code>git checkout -b <branch> origin/<branch></code>	# Create local branch and track remote



Tip:

Always pull the latest changes before pushing to avoid conflicts.

★ Best Practices

- ✓ Always pull before you push.
- ✓ Use meaningful names for branches.
- ✓ Commit small and meaningful changes.
- ✓ Write clear commit messages.
- ✓ Don't push directly to main/master (use PR).

Remember

Remote repositories help in collaboration, backup, and version control.

Keep your remote repo organized and clean!



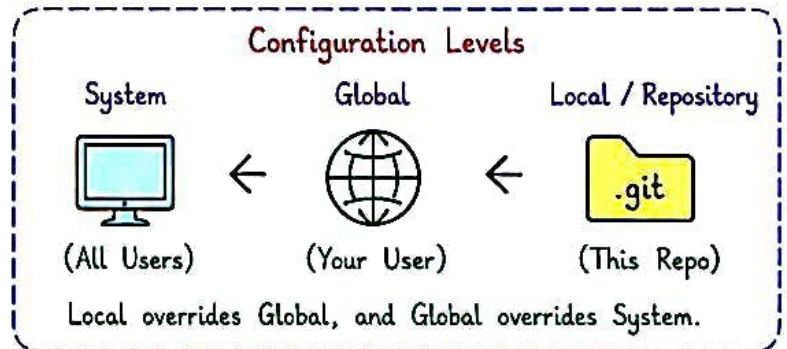
GIT & GITHUB FUNDAMENTALS



19. GIT CONFIGURATION

★ What is Git Configuration?

Git configuration is used to set your username, email, and other preferences. These settings help Git know who you are and how to behave.



★ Set Your Username & Email

```
git config --global user.name "Your Name"    # Set your name
git config --global user.email "you@email.com" # Set your email
```

Example

```
git config --global user.name "John Doe"
git config --global user.email "john@example.com"
```

★ Check Your Git Configuration

```
git config --list    # All config
git config user.name # Your name
git config user.email # Your email
git config --global --list # Global config
git config --local --list # Repo specific config
```



Tip:

Use `--global` for settings that apply to all repositories. Use `--local` for settings specific to a single repository.

★ Edit Configuration File

```
git config --global -e # Edit global config file
git config --local -e  # Edit local repo config file
```

Where are Configs Stored?

- System : `/etc/gitconfig`
- Global : `~/.gitconfig`
- Local : `.git/config` (inside repo)

★ Unset (Remove) a Configuration

```
git config --global --unset user.name
git config --global --unset user.email
```

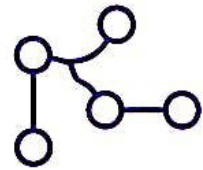
Remember

Always set your correct name and email. It shows up in every commit you make!



★ Best Practices

- ✓ Use meaningful username and email.
- ✓ Keep your email consistent across repositories.
- ✓ Don't share your email if it's private (use GitHub no-reply email).
- ✓ Review your config before committing important changes.
- ✓ Use local config for project-specific settings.



20. GIT ALIASES



What is Git Alias?

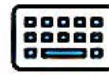
Git Aliases are shortcuts for long Git commands.

They help you save time and make your work more efficient.

Why Use Aliases?



Save
Time



Type
Less



Reduce
Errors



Work
Faster

Aliases make Git more user-friendly!



Create an Alias

```
# Set a simple alias
git config --global alias.co checkout    # 'co' for 'checkout'
# Set another alias
git config --global alias.st status      # 'st' for 'status'
# Set multiple words alias
git config --global alias.br branch -v   # 'br' for 'branch -v'
```

Example Usage

- `git co main`
instead of; `git checkout main`
- `git st`
instead of; `git status`
- `git br`
instead of; `git branch -v`



Useful Alias Examples

Alias	Full Command
co	git checkout
st	git status
ci	git commit
br	git branch -v
lg	git log --oneline --graph --decorate
unstage	git reset HEAD --
last	git log -1 HEAD
type	git cat-file -t
dump	git cat-file -p



Tip:

Aliases are stored in your Git config file:

`~/.gitconfig`

(for global aliases)

You can also set aliases locally for a specific repo.

View Your Aliases

```
git config --global --get-regexp alias
```

Remove an Alias

```
git config --global --unset alias.co
```



Best Practices

- ✓ Use short and meaningful names for aliases.
- ✓ Don't overwrite existing commands you use often.
- ✓ Keep your aliases simple and easy to remember.
- ✓ Use aliases that fit your daily workflow.
- ✓ Share useful aliases with your team.

Remember

Aliases are personal productivity boosters! Use them wisely and make Git work for you!



3. INSTALL GIT (Continued)

Step 1 & Step 2 are completed in previous page.

Windows (Continued)



Step 81 : View commit history.

```
git log --oneline
```

💡 It shows the commit history in a short format.

Step 82 : View remote repositories.

```
git remote -v
```

💡 It lists the remote names and URLs.

Step 83 : Push changes.

```
git push -u origin main
```

💡 It pushes your local commits to the remote repository.

Step 84 : Check status.

```
git status
```

💡 It shows the current status of your working directory.

Step 85 : Add a tag.

```
git tag v1.0
```

💡 It creates a tag named v1.0.

Step 86 : Push tag to remote.

```
git push origin v1.0
```

💡 It pushes the tag to remote.

Awesome! You have learned basic Git operations on Windows. Keep practicing! 😊

Linux (Ubuntu/Debian) (Continued)



Step 81 : View commit history.

```
git log --oneline
```

💡 It shows the commit history in a short format.

Step 82 : View remote repositories.

```
git remote -v
```

💡 It lists the remote names and URLs.

Step 83 : Push changes.

```
git push -u origin main
```

💡 It pushes your local commits to the remote repository.

Step 84 : Check status.

```
git status
```

💡 It shows the current status of your working directory.

Step 85 : Add a tag.

```
git tag v1.0
```

💡 It creates a tag named v1.0.

Step 86 : Push tag to remote.

```
git push origin v1.0
```

💡 It pushes the tag to remote.

Great job! You have mastered basic Git operations on Linux. Keep going, DevOps Hero! 😊

macOS (Continued)



Step 81 : View commit history.

```
git log --oneline
```

💡 It shows the commit history in a short format.

Step 82 : View remote repositories.

```
git remote -v
```

💡 It lists the remote names and URLs.

Step 83 : Push changes.

```
git push -u origin main
```

💡 It pushes your local commits to the remote repository.

Step 84 : Check status.

```
git status
```

💡 It shows the current status of your working directory.

Step 85 : Add a tag.

```
git tag v1.0
```

💡 It creates a tag named v1.0.

Step 86 : Push tag to remote.

```
git push origin v1.0
```

💡 It pushes the tag to remote.

Awesome! You have learned basic Git operations on macOS. You are doing great! 😊

★ Key Takeaways of Day 5

- ✔ Installed Git on Windows, Linux & macOS.
- ✔ Configured username and email.
- ✔ Verified installation and basic settings.
- ✔ Performed essential Git operations.
- ✔ You are now ready to work with Git confidently!



Common Git Commands (Quick Recap)

- | | |
|---------------------|-----------------------------|
| git init | → Initialize new repository |
| git config | → Set user name & email |
| git status | → Check status |
| git add | → Stage changes |
| git commit -m "msg" | → Commit changes |
| git log | → View commit history |
| git remote -v | → View remote repositories |
| git push | → Push changes |



NOTE :

- ★ Practicing daily will make you a Git Expert in no time!
- ★ Small steps today, big DevOps skills tomorrow!

Pro Tip

Always use meaningful commit messages. It makes collaboration easier.



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 5

PAGE : 23 OF 25

3. INSTALL GIT (Continued)

Step 1 & Step 2 are completed in previous page.

Windows (Continued)



Step 87 : Create a new branch.

```
git branch <branch.name>
```

💡 It creates a new branch.

Step 88 : Switch to branch.

```
git checkout <branch.name>
```

💡 It switches to the specified branch.

Step 89 : Create & switch branch.

```
git checkout -b <branch>
```

💡 It creates a new branch and switches to it.

Step 90 : Merge branch.

```
git merge <branch.name>
```

💡 It merges the specified branch into current branch.

Step 91 : Delete branch.

```
git branch -d <branch>
```

💡 It deletes the branch.

Step 92 : Show remotes.

```
git remote -v
```

💡 It shows all remote repositories.

Fantastic! You are building strong Git skills on Windows.
Keep practicing and become a Git Hero! 😊

Linux (Ubuntu/Debian) (Continued)



Step 87 : Create a new branch.

```
git branch <branch.name>
```

💡 It creates a new branch.

Step 88 : Switch to branch.

```
git checkout <branch.name>
```

💡 It switches to the specified branch.

Step 89 : Create & switch branch.

```
git checkout -b <branch>
```

💡 It creates a new branch and switches to it.

Step 90 : Merge branch.

```
git merge <branch.name>
```

💡 It merges the specified branch into current branch.

Step 91 : Delete branch.

```
git branch -d <branch>
```

💡 It deletes the branch.

Step 92 : Show remotes.

```
git remote -v
```

💡 It shows all remote repositories.

Great work! You are doing awesome on Linux.
Consistency makes you a Git Expert! 😊

macOS (Continued)



Step 87 : Create a new branch.

```
git branch <branch.name>
```

💡 It creates a new branch.

Step 88 : Switch to branch.

```
git checkout <branch.name>
```

💡 It switches to the specified branch.

Step 89 : Create & switch branch.

```
git checkout -b <branch>
```

💡 It creates a new branch and switches to it.

Step 90 : Merge branch.

```
git merge <branch.name>
```

💡 It merges the specified branch into current branch.

Step 91 : Delete branch.

```
git branch -d <branch>
```

💡 It deletes the branch.

Step 92 : Show remotes.

```
git remote -v
```

💡 It shows all remote repositories.

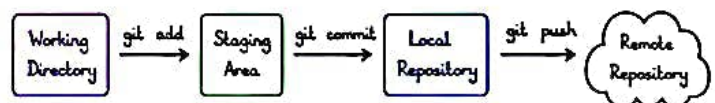
Excellent! You are mastering Git on macOS.
Keep learning and keep growing! 😊

★ Day 5 Summary

- ✓ Installed Git on Windows, Linux & macOS.
- ✓ Configured user name and email.
- ✓ Explored Git configuration settings.
- ✓ Verified installation and basic Git setup.
- ✓ Practiced essential Git commands.
- ✓ Understood Git workflow and branches.
- ✓ One step closer to DevOps mastery!



Git Branching Workflow (Visualizer)



Add → Commit → Push (Repeat this cycle)



NOTE :

- ★ Practice daily to improve your speed.
- ★ Small steps today, big success tomorrow!
- ★ Git is the foundation of DevOps & CI/CD.

Pro Tip



Use "--help" with any command to get more details instantly.



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 5

PAGE : 24 OF 25

3. INSTALL GIT (Continued)

Step 1 & Step 2 are completed in previous page.

Windows (Continued)



Step 93 : Fetch from remote.

`git fetch`

💡 It downloads data from remote without merging.

Step 94 : Pull changes.

`git pull`

💡 It fetches and merges changes from remote.

Step 95 : Stash changes.

`git stash`

💡 It temporarily saves your changes and cleans working directory.

Step 96 : List stashes.

`git stash list`

💡 It shows all stashed changes.

Step 97 : Apply stash.

`git stash apply`

💡 It applies the most recent stash without removing it.

Step 98 : Drop stash.

`git stash drop`

💡 It removes the most recent stash.

Great progress on Windows!
Practice every command and become a Git Expert! 🍀 😊

Linux (Ubuntu/Debian) (Continued)



Step 93 : Fetch from remote.

`git fetch`

💡 It downloads data from remote without merging.

Step 94 : Pull changes.

`git pull`

💡 It fetches and merges changes from remote.

Step 95 : Stash changes.

`git stash`

💡 It temporarily saves your changes and cleans working directory.

Step 96 : List stashes.

`git stash list`

💡 It shows all stashed changes.

Step 97 : Apply stash.

`git stash apply`

💡 It applies the most recent stash without removing it.

Step 98 : Drop stash.

`git stash drop`

💡 It removes the most recent stash.

Awesome work on Linux!
Keep practicing and stay consistent! 😊

macOS (Continued)



Step 93 : Fetch from remote.

`git fetch`

💡 It downloads data from remote without merging.

Step 94 : Pull changes.

`git pull`

💡 It fetches and merges changes from remote.

Step 95 : Stash changes.

`git stash`

💡 It temporarily saves your changes and cleans working directory.

Step 96 : List stashes.

`git stash list`

💡 It shows all stashed changes.

Step 97 : Apply stash.

`git stash apply`

💡 It applies the most recent stash without removing it.

Step 98 : Drop stash.

`git stash drop`

💡 It removes the most recent stash.

Excellent on macOS!
You are almost a Git Master! 🎯 😊

★ Key Takeaways of Day 5

- ✓ Learned advanced Git commands.
- ✓ Understood branch management.
- ✓ Explored merging, deleting & remotes.
- ✓ Practiced stash operations.
- ✓ Now you are ready to handle real-world Git workflows!



Git Command Categories (Summary)

- Setup & Config → config, user name, user email
- Repository → init, clone, remote, fetch
- Basic Operations → add, commit, status, log
- Branching → branch, checkout, merge, delete
- Working with Changes → diff, restore, reset
- Stash → stash, stash list, stash apply, stash drop



Practice + Consistency = DevOps Success!



NOTE :

- ★ Do a little Git practice every day.
- ★ Small steps today, big success tomorrow!
- ★ Git is the backbone of DevOps & CI/CD.

Pro Tip

Use meaningful commit messages.
Example: "Add login page functionality" instead of "Update code"



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 5

PAGE : 25 OF 25

3. INSTALL GIT (Continued)

Step 1 & Step 2 are completed in previous page.

Windows (Continued)



Step 99 : Show branches.

```
git branch
```

💡 It lists all branches in your repository.

Step 100 : Rename branch.

```
git branch -m <new_name>
```

💡 It renames the current branch.

Step 101 : Delete branch (force).

```
git branch -D <branch>
```

💡 It deletes the branch forcefully.

Step 102 : Create & switch
(new in one step).

```
git checkout -b <branch>
```

💡 It creates a new branch and switches to it.

Step 103 : Show commit history
(graphical).

```
git log --graph --oneline --all
```

💡 It shows a compact graphical view of history.

Excellent! You have completed important Git commands on Windows. Keep up the great work! 😊

Linux (Ubuntu/Debian) (Continued)



Step 99 : Show branches.

```
git branch
```

💡 It lists all branches in your repository.

Step 100 : Rename branch.

```
git branch -m <new_name>
```

💡 It renames the current branch.

Step 101 : Delete branch (force).

```
git branch -D <branch>
```

💡 It deletes the branch forcefully.

Step 102 : Create & switch
(new in one step).

```
git checkout -b <branch>
```

💡 It creates a new branch and switches to it.

Step 103 : Show commit history
(graphical).

```
git log --graph --oneline --all
```

💡 It shows a compact graphical view of history.

Awesome! You have completed all essential Git commands on Linux. You are doing amazing! 😊

macOS (Continued)



Step 99 : Show branches.

```
git branch
```

💡 It lists all branches in your repository.

Step 100 : Rename branch.

```
git branch -m <new_name>
```

💡 It renames the current branch.

Step 101 : Delete branch (force).

```
git branch -D <branch>
```

💡 It deletes the branch forcefully.

Step 102 : Create & switch
(new in one step).

```
git checkout -b <branch>
```

💡 It creates a new branch and switches to it.

Step 103 : Show commit history
(graphical).

```
git log --graph --oneline --all
```

💡 It shows a compact graphical view of history.

Great job on macOS! You have mastered the key Git operations. Keep growing, DevOps Hero! 😊

★ Day 5 Key Takeaways

- ✓ Installed Git on Windows, Linux & macOS.
- ✓ Configured user details (name & email).
- ✓ Explored .git directory and basic Git setup.
- ✓ Learned essential Git commands.
- ✓ Understood branching, merging & stashing.
- ✓ Practiced remote operations.
- ✓ One step closer to DevOps mastery!



Git Commands You Must Remember

- | | |
|-------------------------|---------------------------------|
| git init | → Initialize new repository |
| git add . | → Stage all changes |
| git commit -m "msg" | → Commit with message |
| git status | → Show working directory status |
| git log --oneline | → View commit history (short) |
| git branch | → List branches |
| git checkout | → Switch branch |
| git merge | → Merge branch |
| git push -u origin main | → Push to remote repository |

Practice Daily
Be Consistent
Be a DevOps Hero!



NOTE :

- ★ Git is a DVCS (Distributed Version Control System).
- ★ Small daily practice → Big success tomorrow!
- ★ Mistakes are okay, learning is important.
- ★ Git is the foundation of DevOps & CI/CD.

Pro Tip

Use meaningful commit messages.
Good commits make collaboration easier and code history clear.

Reminder



Tomorrow we move to new DevOps concept.
Stay tuned!